DOCUMENT

10

AI Workflow Guide

Context file, skills, agents, hooks and the interaction log

How AI tooling was set up and used on this project: context file, skills, subagents, hooks, spec-driven development and the interaction log.

1. Context file: `AGENTS.md`

`AGENTS.md` is read by coding agents at the start of work (`CLAUDE.md` imports it with `@AGENTS.md`). It holds:

- the project layout and layering rules;
- the approval rules table, boundary values and the two negative rules;
- the commands for tests, mutation and end-to-end runs;
- review expectations: negative tests must prove the invoice is unchanged, fakes must behave like the real thing, every review finding gets exactly one verdict;
- the demo accounts and seed invoices;
- a "Never" list (real credentials, business logic in routers, mutating before all checks pass).

2. Spec-driven development (Spec Kit)

`specify init --integration claude` created `.specify/` and the `speckit-*` skills. The artifacts for this feature are in `specs/001-invoice-approval-limits/`:

Artifact	Purpose
`.specify/memory/constitution.md`	Six principles (rules in the domain, test-first with negative cases, boundaries on both sides, refusals explain themselves, mutation as evidence, small steps)
`spec.md`	User stories, clarifications, functional requirements, success criteria
`plan.md`, `research.md`, `data-model.md`	Design, decisions and rejected alternatives, entities and states
`contracts/approvals-api.md`	The approve endpoint contract and reason texts
`quickstart.md`, `checklists/requirements.md`	How to try it, spec quality checklist
`tasks.md`	Ordered tasks: baseline, foundations, story 1 (single approver), story 2 (two approvers), UI, polish

The artifacts were written directly from the Spec Kit templates rather than by running each `/speckit-*` command in turn.

3. Skills (`.claude/skills/`)

Skill	Use
`approval-rules`	Rule reference and boundary table, loaded when working on approval logic
`review-findings`	The exact findings JSON format and the three allowed verdicts
`run-checks`	Runs pytest, mutmut and Playwright in order and reports one line each
`speckit-*`	Spec Kit commands: constitution, specify, clarify, plan, tasks, checklist, analyze, implement

4. Subagents (`.claude/agents/`)

Agent	Access	Role
`spec-writer`	read, write	Writes or updates Spec Kit artifacts; never touches source
`implementer`	read, write, bash	Implements one task group at a time, tests first, then stops
`test-reviewer`	read-only	Finds vacuous assertions, missing boundaries, negative tests that do not prove "unchanged"
`pr-reviewer`	read-only	Reviews the diff and answers only with the findings JSON array
`mutation-analyst`	read-only	Classifies each surviving mutant (missing test, equivalent, cosmetic) and proposes tests

Project agents are registered only at session start. In the session that created them they could not be selected by name, so the test-reviewer, pr-reviewer and mutation-analyst roles ran as general-purpose subagents told to read and follow their agent files. A fresh session addresses them by name.

5. Hooks (`.claude/settings.json` and `.claude/hooks/`)

Event	Hook	Effect
UserPromptSubmit, PostToolUse (all tools), Stop	`log_event.py`	Appends one JSON line to the interaction log
PostToolUse (Bash, PowerShell)	`check_summary.py`	Turns every pytest, mutmut or Playwright run into `CHECK <name> | metrics | result=pass/fail`
SubagentStop	`validate_review.py`	Blocks a `pr-reviewer` answer that is not the findings JSON schema
SubagentStop	`log_event.py`	Logs the subagent's end

All hooks use only the Python standard library and redact secrets in summaries.

6. The interaction log (`logs/ai-interactions.jsonl`)

One JSON object per line. Every line has the same 27 keys, in the same order, so the log can be filtered and compared. It follows the format asked for in the assignment brief (tool and model, agent details, prompts, skills and sub-agents, tools and integrations, AI actions, human intervention, execution evidence, outcome).

Group	Keys	Meaning
Identity	`id`, `timestamp`, `session`, `stage`, `event`	Unique readable id, UTC time, session, lifecycle stage (specification, implementation, testing, mutation-testing, e2e-testing, review, documentation, oversight ...) and the hook event
AI tool and model	`ai_tool`, `model`	Taken from `.claude/log-defaults.json` (hooks cannot see the model)
Agent	`agent`, `agent_name`, `agent_type`, `mode`	`main` or the sub-agent; `built-in`, `custom`, `sub-agent` or `human`; mode `ask`, `plan`, `agent` or `edit`
Traceability	`related_ids`, `parent_task`, `context`	Each action points to the prompt that caused it
Prompt	`request`, `summary`	The actual prompt (secrets and local paths removed) and a one-line description of what happened
Capabilities	`skills_used`, `subagents_used`, `mcp_tools_used`, `tools_used`	Skills (name, type, purpose, invocation), sub-agents (name, type, task, status), MCP tools (server, tool, purpose) and plain tools
AI output	`suggestion`, `artifacts`	What the agent proposed or reported, and the files it created or changed
Human oversight	`human_decision`, `action_verification`, `limitation`	The person's decision or correction, how the result was checked (the `CHECK` line of a test run), and what went wrong or was left out
Outcome	`status`, `source`	`received`, `applied`, `verified`, `failed`, `completed`, `recorded`; `hook` for live capture, `reconstructed` or `manual` for entries written afterwards

How it is produced: `log_event.py` remembers the latest human prompt, so every tool call, sub-agent report and check that follows carries its id in `related_ids` and `parent_task`. `check_summary.py` turns each pytest, mutmut or Playwright run into a `CheckSummary` record whose `action_verification` is the one-line result. The person's own decisions and corrections are recorded as `HumanDecision` records (`python .claude/hooks/log_event.py --human "what I decided"`).

`python scripts/summarize_log.py` validates every line (same keys, unique ids, every link resolves) and writes the readable summary `logs/ai-interactions-summary.md`: totals, who did what, skills / sub-agents / MCP / tools used, the human decisions, the verification table and the files touched. The hook itself is tested by `python -m pytest .claude/hooks -q`.

The first lines cover planning and the port, which happened before the hooks existed, and are marked `source: "reconstructed"` rather than presented as captured live. Older lines were written with a shorter format and converted to this one, so fields they never held are `null` or empty.

7. Agent workflow used

1. **Plan** the work, ask the user to decide the open questions (FastAPI port, mutmut, Spec Kit).
2. **Baseline:** port the C# code and its 7 tests; record the baseline mutation score.
3. **Context and tooling:** write `AGENTS.md`, skills, agents, hooks.
4. **Spec:** constitution, spec with clarifications, plan, tasks.
5. **Implement in two steps:** single-approver limits, then the two-approver rule, tests first.
6. **Review the tests** with `test-reviewer`; close gaps.
7. **Mutation run** with `mutation-analyst`; add tests for survivors.
8. **Playwright** test for the refused approval.
9. **Review the diff** with `pr-reviewer`; give every finding a verdict and fix the real bugs.

8. Does the context file matter?

A same-prompt comparison (a context-free agent run from an empty directory versus an agent given `AGENTS.md`) is in `ASSIGNMENT.md` section 7. In short: the cold agent got the architecture right; `AGENTS.md` added cent-level boundaries, username normalisation, the Admin rule, exact refusal assertions, real commands and the verdict vocabulary. A first control that ran inside the project was discarded because subagents load project context automatically.

9. Lessons about working with AI here

- Context pays off: with the rules table and negative-test expectations in `AGENTS.md`, generated tests asserted reasons and unchanged state from the start.
- Generated tests looked complete but could not fail until mutation testing and a reviewer agent exposed the forgiving fake and the race tests that passed without the lock.
- The AI review found real defects (a double-approval race, unbounded amounts that broke the list endpoint for everyone). Treat review output as input to a verdict, not as truth: three findings were dismissed with reasons.
- Hooks make the process auditable without extra effort, but a bug in a hook (a regex reading `passed=82 failed=0` as "82 failed") can mislead; hooks need tests too.